PicoCTF

- Verify
    - Quite hard and difficult to understand
    - cat is a new term I learn which means concantenate/ show everything inside the file in terminal
    - sha256 files/* | grep <key>
- CanYouSee
    - Not that hard once I saw the pattern of a 64-bit type url
        - Learned that in cnit176 lab
    - Used exiftool to see indepth information
    - Decode using echo “<URL name>” | base64 -d
        - -d stands for decode
        - Base64 stands for decode in base64 – 128 aplhabets
- Scan surprise
    - Can’t believe it was that easy
    - Only had to scan the qr code which then gave me the flag
- Super SSH
    - Pretty simple but had to get the idea of where to do it
    - SSH basically means secure shell and it can be opened through my own terminal
    - To connect to ssh
        - ssh <username>@<wantedusername> -p <portnumber>
        - Password is <wantedusername’s password>
- Secret of the polyglot
    - Next time, just open the file and look at it.
    - I learned about hexfiles
        - The first few lines would be the file type.
        - Try and compare with different files but in the same folder, you would see a pattern
- BigZip
    - In this challenge I learned how to find a specific file containing a specific item
    - grep -rl “<name of thing you are searching>” filedirectory
    - Then use cat to view the contents in the file
- Unminify
    - This one was solved using web developer/ inspector
    - Just find in there

- WebDecode
    - I finally solved this challenge. It was about finding anomaly in the web inspector.
    - Any anomalies should be considered, unless it is a lot then we can use wget in the command.
    - However, I used an online decoder to solve this problem.
    - There is a way to decode it on the terminal
    - Type *base64 -D* then enter, then put the encoded string
- Interendec
    - This is easy
    - I only had to decode twice using base64 once I figure out it was in base64
    - Then it came to Caesar cipher since the flag was supposed to be starting with picoCTF then it was all browsing
- Repetition
    - Just decode using base64 until completed
- BinHexa
    - Not complicated, basic discrete mathematics
    - Binary shifts, operations, hexadecimal
- Magikarp Ground Mission
    - It's a bunch of directiories that you only had to read
    - The only hard part was connecting to the ssh of the question
- What's a netcat
    - Netcat is basically you're trying to ask your terminal to go somewhere on the internet through a specific port
        - nc <some url> <portnumber>
- Let's warm up
    - When you see 0x60 or anything that number multiplies another number, it would mean it's a hexadecimal and the solution is usually in ASCII
- Don't use client side
    - Open up web inspector if the website is not secured, depending on that, it reveals how you get the flag
- First grep
    - Used grep again.
    - -rl stands for read line
    - But when you don't need it, just do this
        - grep <"phrase you're looking for"> <filename>
- 13
    - Learned about ROT13 encryption

        - It's a substitution of letter depending on the key.
- Warmed Up
    - Hex to decimal
    - Used an online translate
- Glory of the garden
    - When finding a flag cat is a really important tool
- Bases
    - Used base64 encoding
- The Numbers
    - Showed an image with numbers that are arranged in alphabets
- Glitch cat
    - Hex to ascii

Cleared most of the easy part 12 Nov 2024

- Obidient cat
    - Just open it
- 2 Warm
    - Convert to base 2

Note: I stopped doing the daily challenges and only started doing it again on the second week of January 2025

Commitment Issues

- Used git reflog
    - Reference file log, so you can see the versions of the files chronologically.
    - Similar to git log, but a more advanced one
    - Keeps record of changes made to a Git repository's branches, tags and HEAD
- From there copy the HEAD of the version that contains the flag, and paste it with the command *git checkout <HEAD>*
- git checkout
    - switches between different branches in a repository

Password Crack 1 & 2

- Note that python automatically translates Hex to understandable words

- There is no need to decode in this task such as base64 -d <filename>
- Check the python script and find the password there

Runme.py

- Run it and done.

Convertme.py

- Conversion of decimal to binary

Fixme1.py

- Only an indentation error, fix and get the flag

Fixme2.py

- = is not the same as ==. Be careful.

Nice netcat

- Conversion of ascii into text.
- New syntax | tr -d “<something>”
- Translate – delete

Intro to burp

- Port forward to go to next page
- See from unsecured websites

Hashing Job

- MD5 hash encrypter
- Learn more about md5

Tab Tab Tab Attack

- Currently stuck, will find a way in free time
- Shortcut to files in many directories is using tabs

- Use strings <name of file> #very useful
    - Scans a file for sequences of printable characters
- **Wave a flag**
    - Run the commands by following the clues
    - Uses -h (?)
    - An easier way would be to grep
        - Strings warm | grep pico
- **Mod 26**
    - Uses ROT13 encryption
    - Easy to decrypt
    - ROT13
        - Substitution cipher that replaces a letter with the 13th letter after it
        - A == N
        - B == O
- **Static aint always noise**
    - I used the trick from wave a flag
- **Heap 0**
    - Heaps are places in memory and are defined by memory addresses.
    - Heap overflow is when the memory box is full but we try to jam it
    - Heap 0 question wants us to jam it to capture the flag
    - https://medium.com/@dyang./conquering-ctfs-picoctf-heap-0-18bdf49e914f
- **Format String 0**
    - Read the lines of code and understand to do it
- **Blame game**
    - Finding the author using git log
- **Bookmarklet**
    - Code was in javascript, I just had to print it.
- **Collaborative development**
    - Used git log then git reflog to see the heads that changed then git checkout <HEAD> to change versions
- **Codebook**
    - Run the python script and there will be the flag
- **Strings it**
    - I used strings strings | grep pico to find the flag
- **Information**
    - Used exiftool to view the properties of the image.
    - Base64 the licence

- **Cookies**
  - Used web inspector to go to storage and change the cookies
- **Inspector**
  - Played around with the cookies and set admin to true.
- **Vault door training**
  - Password is in the source code
- **Transformation**
  - Two ways to solve
    - Python script
    - UTF8 decode
- **Server-Side Template Injection**
  - Look up pyjail ssti, there's a payload
    - `{{request.application.__globals__.__builtins__.__import__('os').popen('ls').read()}}`
    - Reads the files within that userid
  - Detect using curl -I <url> to see what coding it uses
  - Then go to hacktricks to find ssti
  - Answer to this question is
    - `{{request.application.__globals__.__builtins__.__import__('os').popen('cat flag').read()}}`

LeetCode

- TwoSum
    - Hard, barely understood anything
    - Learned a new concept – hashmaps
- Palindrome
    - Confusing, was not good with palindrome numbers yet
    - But palindrome numbers are numbers, when reversed are the same before it was reversed.
        - 121 reversed is 121- palindrome
        - 21 reversed is 12 – not palindrome
    - Three ways to solve
        - Convert to string, backwards, print
            - #reversed = original[::-1] //this is slicing. By far better

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        original = str (x)
        reversed = original [::-1]
        return original == reversed
```

            - Use reverse function
                - “”.join(reversed(original))

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        original = str (x) #convert to string
        reversed_string = "".join(reversed(original))
        return original == reversed_string
```

- Roman to integer
    - Learn the fundamentals of roman numerals and how to count them
        - If the letter in front is smaller than the number behind, it means to subtract.
        - Else, just add them
    - The basis of this problem relied on that
        - Completed using hash map
        - Compared indexes as such

Roman Numeral	Number

I	1
V	5
X	10
L	50
C	100
D	500
M	1000

```python
class Solution:
    def romanToInt(self, s: str) -> int:
        roman = { "I": 1, "V" : 5, "X" : 10,"L" : 50, "C" : 100, "D" : 500,"M" : 1000}
        result = 0
        for i in range (len(s)):
            if i +1 < len(s) and roman[s[i]] < roman [s[i+1]]:
                result -= roman [s[i]]
            else:
                result += roman[s[i]]
        return result
```

- Add two numbers
    - Numbers are stored backwards
    - This is by far the hardest code I have done so far.
    - Problems include factoring in edge cases and carry numbers.

```python
class Solution:
    def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode])
        dummy = ListNode()
        current = dummy
        carry = 0
        while l1 or l2 or carry:
            v1 = l1.val if l1 else 0
            v2 = l2.val if l2 else 0
            val = v1 + v2 + carry

            carry = val//10
            val = val%10
            current.next = ListNode(val)
            current = current.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None

        return dummy.next
```

  - Will be looking into it again
  - https://www.youtube.com/watch?v=wgFPrzTjm7s

- Longest common prefix

```python
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        #initial answer which is nothing
        res = ""
        for i in range (len(strs[0])):
            for j in strs:
                if i == len(j) or j[i] != strs[0][i]:
                    return res
            res += strs [0][i]

        return res
```

  - Will have to understand better to explain
- Valid parentheses
  - Solution lies in closing the opening of our parentheses (“(“, “)”, “{”, “}”, “[”, “]”)

- Needs to be closed when found an opening, else it is false.
- I used stack, and hash map to solve the problem
- Hash map is used to connect the ending with the starting

}	{
]	[
)	(

- So, we used this hash map through the coding below

closeToOpen = {“}”: “{“} ....

- The rest are solved through stacking or FIFO

```
class Solution:
    def isValid(self, s: str) -> bool:
        #s is the input string whereas self is the
        stack =[]
        closeToOpen = {")":"(", "}":"{","]":"["}

        for c in s:
            if c in closeToOpen:
                if stack and stack[-1] == closeToOpen[c]: #-1 check the las
                    stack.pop() #remove the matching results in true
                else:
                    return False
            else:
                stack.append(c)

        return stack == []
```

- Merge two sorted lists
  - This solution was not easy to come by
    - Used linked list
    - It checks list 1 and list 2 for the smaller value and points the smaller value with the tail

```python
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional
[ListNode]) -> Optional[ListNode]:
        dummy = ListNode()
        tail = dummy

        while list1 and list2:
            if list1.val < list2.val:
                tail.next = list1
                list1 = list1.next
            else:
                tail.next = list2
                list2 = list2.next
            tail = tail.next

        if list1:
            tail.next = list1
        elif list2:
            tail.next = list2
        return dummy.next
```

    - I would need to learn linkedlist deeper.
- Remove duplicates from **sorted** arrays
    - This example uses two pointers
        - Left and right
        - Left measures the distinct numbers in the array
        - Right iterates through the whole array

```python
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        left = 1
        for right in range (1, len(nums)):
            if nums[right] != nums [right-1]:
                nums[left] = nums [right]
                left += 1
        return left #number of output
```

- Remove element
    - There is a number in an array that is the key
    - We want to remove that key in place
    - So values that are not the key, should replace the number
    - However, we only want to return the number of values, but without adding extra memory – in place modification
    - Put non key characters



```
#use a substring
    if needle == "":
        return 0

    for i in range (len(haystack)+ 1 - len(needle)):
        for j in range (len(needle)):
            if haystack[i+j] != needle[j]:
                break
            if j == len(needle) - 1:
                return i
    return -1
```

for j in range (len(needle)):

if haystack [i+j] != needle [j]:

break #this will iterate through the length of the needle and check if the letters are the same